Behavior Catalogs in Practice

Domains as packages, and an agentic loop that formalizes and extends them under a human gate

The metis project*

Draft 0.1 — 2026-10-19

Reading the numbers. Every number in this paper is one the repositories hold: a committed report, test, trace or CI run. A superscript tag such as [S2] names its source; Appendix A gives each tag's file and commit. Nothing was run for this paper. A run reported here is one sample: where a table shows one run per program, it says so.

Abstract

A behavior program is useful only when something can load it, run it and extend it. We describe the *catalog*: one directory per domain, one manifest, one loader that every consumer shares, and a CI that compiles, runs and cross-checks every package in two toolchains. We show the catalog's domains as use cases: story worlds, puzzles, planning, voting, cellular automata, simulated RDDL worlds, and three device domains with hosts and intents. For each family we say what it taught the format. We then describe the *agentic loop*: a model-driven loop that works through a small set of gated tools over the kernel's REPL. It turns a program into a package, exercises it against a simulated world, and proposes extensions; a person approves every write as a diff. We report the loop porting six story programs for 0.06 USD with the programs byte-equal to their seeds. We also report it exercising eight RDDL worlds against pyRDDLGym. The measure is a conditional per-action likelihood, priced on a declared exact or sampled tier, and it records the programs' disagreements with the worlds without tuning them. We close with what the loop could not do, and the rules that forced.

1 Introduction

A language for behavior programs gives a program a meaning: in metis, a catalog of typed, weighted linear-logic rules denotes a probability measure over its own runs [S13]. A meaning is not yet a domain someone can use. Between a program that compiles and a domain a device runs there are several more needs:

- a place for the program's host, its intents and its extensions;
- a name the consumers agree on;
- a start state that both the CI and the live agent read the same way;
- a record of what the program does on a known case;
- a path by which the program changes without anyone editing it blind.

This paper is about that layer. A metis *package* has two consumers: a *loop* that develops it and an *agent* that runs it. Both read it through the same manifest and the same loader (§2). The catalog's CI is the contract between them: every package is run against a committed trace and built into a bundle, and both toolchains must agree on both (§4). The domains

*Sources, catalog, goldens and the loop: `https://github.com/metis-lang-dev` (metis-catalog, metis-loop, metis-agentic-loop, metis-agent, metispy, metis-lang). Correspondence: `info@robofortis.com`.

themselves, 29 packages today, are the use cases. They are what the format had to absorb (§3).

The second half is the *agentic loop* (§5): a language model working the kernel's REPL and the domain agent through generated tools. Its one capability that neither interface has is writing into the catalog, and every such write is a diff a person approves. That gate is the invariant this paper keeps returning to. Two case studies report what the loop did with it: porting six story programs (§6), and exercising eight simulated worlds whose measure needed the watch itself to change (§7). Section 8 lists what the loop could not do and the rules that forced.

The stack. metis has three layers and three papers. The *language* paper, *Behavior Programs as Measures over Proofs* (draft 0.3), gives the semantics, the compiler `metisc`, the referee `metispy`, and their parity [S13]. This paper is the second layer: the catalog and the loop. The third, *Small Models, Certified Behavior* (in preparation), is the agent: a small local model made domain-competent by a certified runtime.

2 The package

A package is a directory, `domains/<name>/`, with a manifest, `domain.yaml`. The package's name is its directory's. Every manifest key is optional, and the loader refuses any key it does not know [S1]. The keys:

- `program`: the program file;
- `extensions` and `serves`: the served program;
- `host`: a Python factory for the world;
- `intent`: the spec of the person's verbs;
- `clock`: env beats per tick, and the watch's settings (§7);
- `records`: where surprise records go;
- `reload`: the agent's reload policy when the package's files change (propose, or auto);
- `display`, `line`, `voice`: the narrator's name, the console line, the voice;
- `agent`: a domain-agent module, when the generic one does not fit;
- `pair`, `partner_of`: two cooks stepped in one env;
- `notch`: a fitted overlay of weights.

One loader, `metis_catalog`, reads every manifest for every consumer: the domain agent (Python and Go), the loop's SDK, the two consoles and the CI.

Figure 1 shows the dog, the reference device domain. Its program is 286 lines; its host is a Python file; its `intent.yaml` is written by people. Everything under `intent/` is generated from it [S1]:

- the grammar the intent pass is held to (`dog.gbnf`);
- the prompt;
- the spoken examples;
- the corpus.

Extension, pack, bundle. Three words, kept apart [S1].

- An *extension* is a catalog in the package's `extensions/` that `extends` the program. The served program is the program plus its extensions in name order. The plant's `extensions/visitor.llp` is one: a rule for a visitor who waters the plant.
- In the REPL, a gated candidate is a *pack* (the language's word). Once accepted into a package it is written as an extension.

```
domains/dog/
  domain.yaml        display: the dog
  dog.llp            line: "a robot dog: walks to places, finds things,
  dog_host.py                sits and stands"
  intent.yaml        voice: own
  intent/            program: dog.llp
    dog.gbnf         host: dog_host.py:make
    dog-prompt.txt   intent: {spec: intent.yaml}
    dog-spoken.txt   clock: {beats: 1}
    dog-corpus.tsv   records: records/
  README.md
  dog-transcript.md
```

Figure 1: A package: the dog's tracked files and its manifest (metis-catalog `domains/dog/`, at 07d232b [S1]). `intent/` is generated from `intent.yaml`; `records/` is written by the agents.

- A *bundle* is what the emitter forms from a package for the C++ and Go runtimes: `ground.wire` and `sidecar.json`, with `program.ir` and `tools.json`. The OCaml compiler writes `ground.wire` (`metisc --wire <program> [--extension e.llp]...`). metispy's emitter must produce the same bytes; the catalog's CI checks this for every package (§4).

The start is the program's. A program's `#init` lines are its live start: the domain agent, the consoles and the watch all read them, and `#steps` is the watch's horizon. So a line added for CI must not change the live behaviour. When a package's own start fires nothing, its CI case lives outside the package, in `goldens/cases/<name>.llp`: the program included unchanged, with a start that fires (§4) [S2].

3 The catalog as use cases

Table 1 lists the packages by family, with what each uses of the language. At metis-catalog f25c758 the catalog held 28 packages. kitchen_shared is counted there and listed under kitchen below, as the loader lists it. road became the 29th at 07d232b [S2,S16].

3.1 Story worlds

The six story programs are ports of Ceptre examples and their follow-ups: love_triangle, tragedy, herstory, talespin, three_pigs and three_pigs_connectives. The loop brought them into the catalog (§6). They are what the format had to absorb for programs with no device behind them:

- a domain with no intents must still open (§8);
- a case with an `#interactive` choice must run under CI with no terminal, the sampler choosing;
- a ground can be large: talespin's has 105,680 events, and `metisc` grounds it in about 130 s, nearly all of the packages job's time [S2,S4].

Figure 2 shows the smallest: three_pigs' case and its committed trace.

family	packages	what they exercise
story worlds	love_triangle, tragedy, herstory, talespin, three_pigs, three_pigs_connectives	Ceptre ports; `#interactive`; multiplicities; weights
puzzles, planning	wgc, bridge, bridge_decl, blocks_world, tower_of_hanoi	a host-side solver (wgc); stages and links; `#interactive`
small formal worlds	primes, ll, cellular_automata, gossip, fptp	linear logic as computation; stages
device domains	dog, plant, kitchen (+ kitchen_shared)	hosts; intents; an extension; ports
simulated worlds	nav, hanoi, gol, sysadmin, xtraffic, elevators, wildfire, tamarisk	a gym host; weight ports; two notches; stage pipelines
parked	road	the production road catalog on one scene

Table 1: The 29 packages at metis-catalog 07d232b, read through the loader's own index (kitchen_shared is listed by its lead) [S1,S16]. Hosts: the device domains, wgc and the eight simulated worlds. Intents: dog and plant. An extension: plant. A notch: wildfire, tamarisk.

```
#init pig 3
#init wolf 1
#init straw 1
#init sticks 1
#init bricks 1
#steps 8
#seeds 1
#query strawhouse
#query stickhouse
#query brickhouse
```

```
case three_pigs steps=8 seeds=1
I pig 3
I wolf 1
I straw 1
I sticks 1
I bricks 1
T 1 build_stick build_straw blow_straw
    build_brick blow_stick blow_brick
F 1 (done) brickhouse:1
Q strawhouse 0/1
Q stickhouse 0/1
Q brickhouse 1/1
Q wolf 0/1
```

Figure 2: A case and its trace: `domains/three_pigs/three_pigs.llp` lines 49–58, and `goldens/traces/three_pigs.txt`, which the CI replays byte for byte (the `T` line is wrapped here) [S15]. One seed: one sample of the program's measure, not its law.

3.2 Puzzles and planning

wgc (wolf, goat, cabbage) is the one puzzle with a host. Its domain agent searches the program's moves with a host-side greedy solver, and the domain agent's step cap exists because of it: a program that never settles cannot hold the clock (§5). bridge and bridge_decl are two formalizations of one statement, bridge and torch:

- bridge is imperative;
- bridge_decl holds minute-marks and pace as data.

They are two packages, not a program and its extension: bridge_decl does not `extends` bridge, and a package serves one program [S2]. blocks_world and tower_of_hanoi are Ceptre examples. tower_of_hanoi coexists with the RDDL `hanoi`, and both READMEs say which is which.

3.3 Small formal worlds

primes is a sieve: every number annihilates its multiples. ll is a toy linear-logic prover. gossip spreads posts in a small network. fptp counts ballots. cellular_automata is an honest skeleton: the original has no rules, so its committed trace is empty [S2]. These are linear logic as computation. They taught the format that a package need not be a "domain" in any device sense: a program with a case is enough.

3.4 Device domains

dog, plant and kitchen are the domains the agent speaks for. Each has a host (the world), and dog and plant have intents. kitchen has a domain-agent module of its own, and its pair, kitchen_shared, is stepped in one env with it. plant carries the catalog's one extension, `extensions/visitor.llp`: a visitor waters the plant, which the program alone has no rule for. These three fixed the manifest's shape [S1]. Their own starts fire nothing (no command is held), so their CI cases are in `goldens/cases/` (§4) [S2].

3.5 Simulated worlds

Eight packages are worlds from the IPPC/RDDL benchmarks, run against pyRDDLGym. In each, the gym is the world and the program is the formalization:

- nav, hanoi, gol, sysadmin, xtraffic and elevators come from metispy's models or the RDDL headers;
- wildfire and tamarisk were formalized from gym episodes in an earlier metis loop, and keep the notch fitted then.

Each package's host is a thin configuration over one class, `tools/rddl_host.py`, which owns:

- the gym child process;
- the fluent-to-atom mapping (`elevator-at-floor___e0__f2` is the atom `elevator_at_floor(e0,f2)`, unless the world maps its own);
- the mapping from fired rules to gym actions;
- the snapshot and restore the REPL's forks need [S7].

There are two kinds of world:

- in an *acting* world (nav, hanoi, xtraffic, elevators) the program's rules choose the moves and the gym answers;
- in an *observing* world (wildfire, tamarisk, and since rddl-watch gol and sysadmin) the host acts, either a seeded random agent or a no-op, and hands the step to the program, whose pipeline computes the next state [S7,S8].

Section 7 is about measuring them.

3.6 What stays outside packages

Some programs are not packages:

- the RDDL experiments' records (`domains/rddl/`: born, candidate and iteration worlds);
- the port-plugin conformance fixtures;
- the C++ runtime's acceptance demo;
- the CI cases.

They keep their roundtrip and diagnose gates. The road catalog (chariot's production road-behavior catalog, its scenes and its packs) was outside until road-collapse made it one parked package. Its program is the production catalog included unchanged on one scene, one tick, so that it runs and builds nominally. Its real port, with SUMO as the runtime, is the next release's subject [S16].

gate	what it checks	time
`check.sh`	roundtrip; canonical keys (scenes → `KEYS.txt`)	0.3 s locally
`diagnose.sh`	`metisc` diagnostics, 72 catalogs, exemptions by reason	0.6 s locally
`crosscheck.py`	metispy compiles what `metisc` compiles	0.2 s locally
`packages.py`	run, twin, wire: 92 gates over 32 units	141 s locally, 2 workers
job `packages`	the above on the public runner, toolchains built	3 min 53 s

Table 2: The catalog's CI at metis-catalog f25c758 [S2,S3]. The `packages` job's runner time is its CI run at f25c758. At 07d232b (road included) `packages.py` ran 33 units in 256 s on the runner, in a 4 min 46 s job [S3].

4 CI as the catalog's contract

The catalog's CI has two jobs (Table 2).

- `catalogs` compiles every `.llp` file, packages or not:
  - a roundtrip;
  - `metisc`'s diagnostics, with each exemption listed with its reason;
  - the canonical-key gate: scene instances ground to committed keys;
  - a crosscheck that metispy compiles every file `metisc` does.
- `packages` gives each package three gates [S2]:
  - *run*: `metisc --run` of the program's `#` case, stdin closed, equals the committed `goldens/traces/<name>.txt` byte for byte;
  - *twin*: metispy's replay of the same case gives the same start, fired and final lines;
  - *wire*: `metisc --wire` of the served program equals metispy's bundle `ground.wire` byte for byte.

  The cases in `goldens/cases/` get the run and twin gates. Every tracked package needs a trace, and every trace a package or a case.

Before the CI pinned anything, both toolchains had to be shown deterministic. `metisc`'s `--run` seeds the reference sampler with the seed index; run twice, it was byte-equal on all 28 packages, and metispy's replay gave the same start, fired and final lines [S2]. The query lines are hit counts over the seeds, and metispy annotates them with exact probabilities that `metisc` does not print. So the twin compares the other lines only, and nothing pins noise.

Live-neutral lines, and cases. A program's `#init` is its live start (§2). So only lines that leave the live behaviour alone may be added for CI: `#steps 6` (the watch's default horizon) and `#seeds`. `#query` is not live-neutral: the console's mirror projects every query on each call [S2]. Where a package's own start fires nothing (dog, plant, kitchen, sysadmin), its CI case is a separate file. That file includes the program unchanged, gives a start that fires, and has a trace of its own. The cost is honest and stated: dog's case repeats its program's 32 `#init` lines, because directives are read from the file that is run, not from its includes [S2].

A bug the CI script had. The first `packages.py` ran its gates in threads, and read `metisc`'s output through a pipe. While a thread computed talespin's Python twin, it held the interpreter lock and starved the pipe's reader. `metisc --wire`, 56 MB of output, stalled for over 13 minutes, against 119 s alone. Each gate now runs in its own process, writing to a temporary file [S2].

state	what happens	tools
seed	a seeded start: the REPL serves an existing program, which becomes the domain's program as it is, gated	`catalog_apply` served
formalize	prose into the REPL; the kernel's formalize drafts the program, checked whole	`repl_text`, `repl_formalize`, `repl_revise`
apply	the candidate written into the catalog through the SDK's one validated write, gated	`catalog_apply` program
intents	the person's verbs drafted from the program's `cmd_*` atoms, gated	`catalog_apply` intents
exercise	the domain agent runs the package, the loop owning the clock	`domain_open`, `domain_clock`, `intent_<verb>`
exercise (world)	a world with no intents, measured through its ports and seeded episodes; recorded, not fixed	`domain_ports`, `domain_episodes`
surprise	the world reads differently from the program; a message enters the loop	(a signal)
revise	the missing behaviour drafted as an extension and gated	`repl_mine`, `repl_gate`
edit	one exact edit of a domain's file, every other byte kept; never formalize	`catalog_edit`
take	the finished domain taken into the person's catalog, gated, never a commit	`catalog_take`

Table 3: The loop as a state machine (abridged from `docs/LOOP-AGENT.md` [S9]). The person decides every write into the catalog: `catalog_apply`, `catalog_edit` and `catalog_take` each wait at a gate that shows the SDK's dry-run diff.

5 The agentic loop

The agentic loop is a language model, hosted in pi, working two processes through generated tools [S9]:

- the kernel's REPL (metispy, over its JSON-lines wire);
- the domain agent (metis-agent's lower half, which runs a package with its host and intents).

The loop's tools are generated from those two interfaces' own declarations. Nothing is written for pi that the interfaces do not declare, and nothing they declare is left out. pi's built-in tools, context files, skills, prompt templates and MCP are all off.

Table 3 is the loop as a state machine. Three kinds of tool matter:

- **the REPL's verbs** (`repl_state`, `repl_show`, `repl_text`, `repl_formalize`, `repl_revise`, `repl_gate`, …), one per verb the wire lists;
- **the domain agent's faces** (`domain_open`, `domain_clock`, `domain_state`, `domain_events`, `domain_ports`, `domain_world`, `domain_episodes`), and one `intent_<verb>` per verb of the domain's `intent.yaml`;
- **the three writes** into the catalog, which neither interface has:
  - `catalog_apply` writes a program, an extension, a served seed, a README or the

```
--- a/domains/plant/extensions/visitor.llp
+++ b/domains/plant/extensions/visitor.llp
@@ -0,0 +1,6 @@
+catalog visitor 1.
+extends plant.
+stage main {
+  %% A visitor has watered me; my soil is moist again.
+  visitor_waters [dyn] : dry * visitor -o moist.
+}
\ No newline at end of file
approve? [y/n]
```

Figure 3: A gate: the loop SDK's dry-run diff (`python -m metis_loop_sdk.catalog apply plant --extension visitor=... --dry-run`) of the committed `domains/plant/extensions/visitor.llp` [S1,S10]. The `%%` line is what the device says when the rule fires. The last line is the gate's question; the person answers it.

intents;
- `catalog_edit` makes one exact edit of a file, old text to new;
- `catalog_take` brings a finished domain into the person's catalog.

All three go through metis-loop's SDK, which has one Python face and one TypeScript face at one tag.

The gate. Every write waits at pi's `tool_call` gate. The gate shows the SDK's dry-run diff, and the person answers yes, or no with a steer. Two details make the gate readable rather than decorative [S9,S10]:

- a file the person refused shows, at its next proposal, the diff from the refused text first, then the diff against the disk;
- a README the person refused is never written: it comes back through the same gate.

The extensions' code decides nothing about the domain. It spawns the processes, generates the tools, passes calls through, turns unasked lines (events, quiescence, surprises) into messages, and refuses an apply whose candidate has not passed its gate. The model decides the order of calls and what a surprise means. The person decides every write, and steers and stops.

Figure 3 is the shape of a gate: the SDK's dry run of the plant's visitor extension, applied to a copy of the plant without it.

The seed task. A seeded run makes an existing program a package. The seed task has six steps [S9]:

1. read the program;
2. apply it as served;
3. the intents, or none;
4. exercise it;
5. a README;
6. a take, when the person asks.

It states the rules that the case studies below forced:

- no formalize or revise of a seed, ever: they reprint the program and drop its comments;
- a change the person asks for is a `catalog_edit` with the exact old and new text;
- every claim in a README needs a source the model can name;
- one run is one sample, never "always" or "only";

program → domain	time	req.	key USD	gates ok/refused	program diff	events
love_triangle	107 s	13	0.0054	5 / 1	none	27
tragedy	123 s	19	0.0103	5 / 1	none	14,800
herstory	86 s	16	0.0130	4 / 0	none	23
talespin	89 s	14	0.0075	4 / 0	none	105,680
three_pigs	129 s	17	0.0114	4 / 3	none	6
three_pigs_connectives	225 s	21	0.0120	4 / 2	1 comment line†	7

Table 4: The six ports, one run each (fiction-port step 2 [S4]). Cost is by the API key's usage, which lags about a minute, so the per-run deltas do not sum exactly. The whole subject was 0.0624 USD by the key, against a budget of 1.5. Refused gates were READMEs, except one program gate (finding 1). Every program is the seed byte for byte. †One comment line in three_pigs_connectives was edited by hand, because the loop then could not. "Events": the bundle's ground.

- a world without intents is measured with the person's n, set before the task begins, and recorded, never fixed.

6 Case study A: porting six story programs

The six Ceptre-derived story programs were ported by the loop, one run each (`loop.sh --seed`, deepseek-v4-pro on OpenRouter), and taken into metis-catalog with `catalog_take` [S4]. Before every approval, the person checked mechanically that the gate's program equals the seed byte for byte. At each take gate, the person checked that the README and intents equal the versions already approved. Table 4 gives each run.

The order of calls held every time: served, intents, open, clock, state, events, README, take, with no wrong tool [S4]. No take was refused, and every take carried exactly what had been approved. The findings, ranked as reported, are where the gate earned its place [S4]:

1. **The loop could not edit a program's comments, and the model's try stripped all of them.** Asked to change one path in a header comment, the model ran `repl_text` with the whole program, then formalize, revise and apply. The candidate was the kernel's canonical print: every rule line identical, every `%` comment gone (a 44-line header with origin and lessons). The full-diff gate caught it, and the person refused. The fix was `catalog_edit`, an exact old-to-new edit through the SDK that never passes through formalize [S10].
2. **A README resent for a one-sentence fix came back with new errors elsewhere:** typos, a misspelt file name, and a broken run command. Because the README was never applied, the gate showed it as new each time, and the person caught the damage only by diffing against the refused version. The fix was the gate's diff from the refused proposal [S10].
3. **Invented facts in READMEs.** Four of six READMEs needed a refusal despite the instruction "copy those, invent none". One misquoted the author. One claimed "only the brick house survives", which is false when the wolf reaches the brick house first, and generalizes from one sampled run. One misquoted the program's own catalog name. The fixes: the SDK writes a README's mechanical parts from the files (origin, header, how to run, intents), and the model writes one paragraph in which every claim needs a source [S9,S10].
4. **Gaps in the seed task**, which cost every run extra reads.

The plain summary is the report's own: the gate held every time, and a README a model writes must be read like code [S4].

die weight	per-beat log-likelihood	mean log P(answer | action)	actions scored
×1 (the instance)	−1.104	**−0.199**	52
×0.2	−0.738	−0.219	138
×5	**−0.168**	−0.493	27

Table 5: nav, seeds 1–20: the per-beat measure ranks the wrong rate best; the conditional measure ranks the instance's rate best (rddl-worlds step 1 [S5]). The ×0.2 gap is small enough that other seeds could close it.

7 Case study B: exercising simulated worlds

A world without intents is exercised through its world. Its ports name the gym, the instance, the observed atoms and the actions. `domain_episodes` runs n seeded episodes. Each is a reset with its seed, on both the program's sampler and the gym, with every beat scored to the episode's end [S9].

The measure. Score a beat by the program's probability of the world's reading, and it prices the program's own choice of action. A wrong rate can then look better. On nav, seeds 1–20, the per-beat likelihood ranked a die rate five times the instance's best (Table 5) [S5]. The measure the loop uses is conditional instead: per action the program sent, P(the world's answer | that action). That is the weight of the same-action alternatives in the drawn tier whose effect is the world's reading, over the weight of them all. It is RDDL's own transition model, scored only when the program's state agreed with the world's before the action. It ranks the true rate best. Each variant is still scored on its own trajectories, so the scored-action count is reported beside every mean.

A planted wrong rule is caught. The surprise test is the kernel's, unchanged: a reading the served program gives zero probability opens a record. With nav's die rules deleted, 18 of 20 episodes surprise: the robot vanishes and the program cannot say so [S5]. On gol, with the rule for a dead cell coming alive deleted, the test requires surprises in at least 3 of 5 episodes, each naming a birth [S14].

The lockstep problem. The first pass over the seven new worlds (rddl-worlds step 3) measured nav and hanoi fully, but none of the other six [S7]. Six of the seven compute one gym step as a cycle of many rows, and the watch read the world after every row. That went wrong in four ways:

- **a mid-cycle read surprised.** gol's mark pass consumes a cell the gym still shows; xtraffic's gym answers a move with the obstacles already shifted, which the program reaches 13 rows later.
- **a pipeline was not restarted at a tick.** wildfire and tamarisk stalled at their second step.
- **exact pricing of a 78-row cycle was too slow.**
- **after a mismatch the program could still explain, the watch never re-checkpointed.** elevators went from 0.1 s to 22 s a row by step 12.

Two worlds also never really started. sysadmin had no start token, and elevators' `#init` was not the gym's reset state. And one engineering failure belongs in the record. A host snapshot that grew with every fired event, stored once per row, took a diagnostic to 23.7 GB in two minutes. The snapshot now holds the episode only, bounded by the horizon [S7].

world	surprise episodes, step 3	after rddl-watch	mean log / beat	time (20 ep.)
nav	0	0 (= step 3)	−0.022	9 s
hanoi	0	0 (= step 3)	0.000	8 s
sysadmin	0 (never fired)	0	−0.491	9 s
gol	20, row 1	0	−0.974	55 s
xtraffic	20, first move	0	−0.584	10 s
elevators	20, row 0	20, gym steps 1–33	−0.898	13 s
wildfire	20, beat 1 (stalled)	0	−0.477	302 s (957 s)
tamarisk	20, beat 1 (stalled)	0	−2.036	592 s

Table 6: Eight worlds, 20 seeded episodes each, deterministic: rddl-worlds step 3 against rddl-watch [S7,S8]. "Mean log / beat" is the per-beat log-probability of the gym's reading, averaged over beats with a nonzero estimate. nav and hanoi reproduce step 3's numbers key for key. wildfire's and tamarisk's passes are sampled; gol, sysadmin, xtraffic and elevators are exact. xtraffic's 800 actions are scored, none at $p = 0$; its mean log per scored action is 0.000 (every answer had $p = 1$), so there the measure counts agreement rather than ranking rates; its reward went from −40 to −15.3, because its robot now moves through the episode. Every run was under a 6 GB memory cap; peak RSS 80–104 MB.

The fix, behind the manifest's clock (rddl-watch). Every change is a setting a package declares. Without it the watch is unchanged, and the Python and Go agents stay byte-equal on their recorded days [S8].

- `clock: {lockstep: true}` (gol, xtraffic, sysadmin, wildfire, tamarisk) reads the world once per pass of the stage pipeline. The pass is done when it wraps (a link back into the first stage) or when nothing is left to fire; both are read off the program's own stages, with no per-world constant. A wrapped pass waits for the beat, and the beat's input restarts a quiet pipeline at its first stage. Every beat is the window's boundary.
- `clock: {follow: true}` (elevators, a one-stage program that never rests) adopts a reading the program explains as one exo row, and moves the checkpoint there. With its host reading only after a row that stepped the gym, a window runs from one gym step to the next.
- sysadmin's host hands its start token when the program lacks it, and every RDDL world now starts at the gym's reset state, through the same exo row the world lines use.
- The Go agent refuses both settings by name.

The tier. The exact filter is the default. A window whose exact filter outgrows 10,000 cumulative retained states goes to a sampled tier: 500 reference-sampler runs, seeded from the episode's seed and the session's position, with a 95% Wilson half-width [S8]. Every record and score declares its tier, and the sampled tier's runs are the reference sampler's own draws, indexed once per window. gol's pass needs 6,665 states and stays exact. wildfire's and tamarisk's need more than 300,000, so they are sampled. A sampled zero is no proof: it means no run of 500 reached the reading, below the tier's resolution ($p < 0.0076$ at 95%). So it is kind *rare*, never a surprise, and the loop counts it apart [S8,S9]. Tamarisk shows why: four of its zeros at $n = 500$, re-drawn at $n = 20{,}000$, were $p \approx 0.0065$, 0.0015, 0.0001 and below 10^{-4} [S8].

Table 6 is the result. The surprises the watch itself produced are gone. What remains is the programs' honest disagreement, recorded for a tuning subject and not fixed [S8]. Elevators surprises in every episode, at gym steps 1–33, in two ways:

- in 18 episodes, an arrival falls in a gym step whose window held only the action row:

the program draws arrivals as rows of their own between actions, while the gym draws them inside its step;
- in 2 episodes, the gym boards a passenger at `close`, where the program boards only while the door is open.

The most mismatched atoms are expected sampling elsewhere, except xtraffic's obstacle arrivals at the road's edge.

Through the loop, and the cost. A real loop run read the same measure: one pi run on gol through `domain_open`, `domain_ports` and `domain_episodes`. Its 20 episodes gave the domain agent's numbers (820 beats scored, mean log −0.974, no surprise), both gates were byte-equal, and the take was identical [S8].

- That run cost 0.017 USD by pi's count, 0.051 by the key's usage over its window [S8].
- nav's loop run (rddl-worlds step 2: the package taken byte-equal, its measure the domain agent's) cost about 0.006 USD by the key [S6].
- The nine loop runs of rddl-worlds step 3 came to 0.221 USD by pi's count and 0.125 to 0.154 by the key [S7].

No run tuned anything: no weight, no notch, no extension.

8 What the loop could not do, and the rules it forced

Each finding below changed a component, and each is a commit.

- **A program-only domain must open.** The story worlds have no intents and no host. The domain agent now builds them as bare domains (an empty card, no plugin) in Python and Go. Every intent then gets the same refusal in both, "this domain takes no intents". A new recorded day holds them at parity: 27 output lines equal [S11].
- **`metisc --run` on an `#interactive` case with stdin that is not a terminal** spun at full CPU and printed nothing. After end of input, every later choice is now the sampler's, and the case header is flushed before the ground [S4,S11].
- **A take must be byte-preserving.** Elevators' header, kept verbatim from the RDDL file, came back with LF line ends instead of CRLF. Take, plan, apply and edit now keep a file's bytes [S12].
- **A notch is carried, never edited.** Take did not carry a manifest's notch, so wildfire and tamarisk could be neither served nor taken. It does now, and an edit of a notch is refused: no loop tool writes weights [S12]. Porting and exercising have one budget; tuning has another, and a separate subject.
- **The model ran $n = 20$ before the person's cap arrived**, twice: in rddl-worlds step 3 and again in the gol run. The person's n is now set before the task begins (`loop.sh --episodes`), and the seed task carries it [S9].
- **The SDK serves a program only as `<name>.llp`,** and the model misread the refusal for a whole settle. The refusal now says what to do [S7].

And the person's own rule, which no component enforces: approve only the diffs you have read. Twice, the person's check (a byte comparison, or a diff against the refused version) found what the model's account of its change did not say [S4].

9 Related work

The story worlds are ports of Ceptre, Martens' linear-logic language for interactive narrative, and its examples. metis keeps Ceptre's multiset-rewriting reading and adds a measure [S13]. The simulated worlds are IPPC benchmarks, written in RDDL and run in pyRDDLGym; here

the gym is the world a formalization is measured against, not a planner's environment. The agentic loop sits among coding agents that edit code with tool calls under human approval, and among program synthesis with execution feedback. What it adds is narrower: every write goes through one validated SDK and a person's diff; the loop's tools are generated from the interfaces it drives; and it measures a program against a world rather than against tests. We make no comparative claim. This paper reports one system's runs, each one sample.

10 Conclusion and next

A program becomes useful as a package: one manifest, one loader, a committed case and trace, a bundle both toolchains agree on, and a CI that says so for every package on every push. The agentic loop grows packages under a gate a person reads. In the two case studies it ported six programs byte for byte for 0.06 USD, and it exercised eight worlds to a measure that records disagreement instead of hiding it.

Three subjects come next.

- **Tuning.** A tuning subject, budgeted apart, starts from Table 6. The open question is scoring models on one fixed set of recorded transitions, rather than on each model's own trajectories [S5].
- **SUMO for road.** The road domain gets SUMO as its runtime, and a real port [S16].
- **Intents for worlds.** Worlds that take commands get intents.

A Sources

Reports are subject files (`subj cat <sid> <path>`) that name the commits they report on; commits are in the metis-lang-dev repositories.

tag	source
S1	metis-catalog `CATALOG.md`, `README.md`, `domains/dog/`, `domains/plant/`, the loader `metis_catalog`; at 07d232b
S2	catalog-packages s-a2c529d6 `steps/step1-A-report.md`; metis-catalog f25c758 (`ci/packages.py`, `goldens/traces/`, `goldens/cases/`)
S3	metis-catalog CI: run 39234382318 at f25c758, job `packages` 10:01:15–10:05:08 UTC; run 39265998445 at 07d232b, "33 units in 256s", job 14:06:47–14:11:33 UTC
S4	fiction-port s-e3d0942d `steps/step2-report.md`; metis-catalog 335e637
S5	rddl-worlds s-242c7c3f `steps/step1-report.md` and `steps/step1-opus-review-B.md`; metis-agent 292d6ef (the scored beat), metis-agentic-loop 1795517
S6	rddl-worlds s-242c7c3f `steps/step2-report.md` (nav through the loop)
S7	rddl-worlds s-242c7c3f `steps/step3-report.md`; metis-catalog d821b9c, 28a3900; metis-agent be00c35
S8	rddl-watch s-cfe94dca `steps/step1-report.md`; metispy d33b779, metis-agent 41c5ed7, metis-catalog 4f0d3ec, metis-agentic-loop 1718e0d
S9	metis-agentic-loop `docs/LOOP-AGENT.md`, `prompts/seed-task.md`, `extensions/`; at e6026c1 (the sampled tier counted apart, `--episodes`)
S10	loop-fixes s-855d8067: metis-loop 5fe077c (the SDK's edit, the since-diff, the README's mechanical parts), metis-agentic-loop 73d770e, e31fe33, 04506d2
S11	agent-no-intent s-7fd29000 `steps/step1-report.md`; metis-agent a0a19e6, metis-catalog e1f7607, metis-lang 2651103
S12	metis-loop f562644 (bytes kept), e1bfcc7 (the notch carried), 0ba8d84 (a notch never edited)
S13	metis-lang `paper/metis.tex`, draft 0.3
S14	metis-agent `py/tests/test_rddl_world.py`, `py/tests/test_rddl_watch.py`; at 41c5ed7
S15	metis-catalog `domains/three_pigs/three_pigs.llp`, `goldens/traces/three_pigs.txt`; at 07d232b
S16	road-collapse s-2a2cbea3 `steps/step1-report.md`; metis-catalog 115a8b8, 07d232b